Searching below 208 states: a comparative audit and reduction search

Binary Turing machines for the consistency of Peano arithmetic

29 October 2026

Abstract

The updated task is to find a justified construction with fewer than the 208 working states already supplied in two later commits. This dossier does **not establish a 207-state machine**. It compares the proposals, freshly replays both final 208-state machine classes, and records additional research with reproducible scope and explicit proof boundaries.

The five U208 packets contain nine presentations of only two machines. Every packet uses the same conditional-history identification of native states 172 and 211; their savings cannot be added. Beyond that existing fold, we prove a finite-model obstruction to globally identifying the cover emitter's source and output roles, exhaust all 279,936 independent parameter permutations in seven axiom rows (minimum source DAG size 197), and search broader local reductions, including pairs whose two outgoing transitions both change. No accepted additional deletion is claimed. The accompanying code, exact inputs, receipts, and rejected examples make the successful checks and negative search results independently reviewable.

Latest requested source snapshot. `34aaab66fc5dc834863ed00ea84823c93b8c5b1e` in `VladimirReshetnikov/ProveIt`. The earlier requested U208 snapshot is `ef5a23e04b0285e321ceec5f4b329223cee0ab57`; the original U209 comparison uses `fe8b3a2d1db0033113e986590f79ecba3e5ef500`.

Reading guide. Sections 2–3 identify the supplied machines and the exact existing saving. Section 4 separates the machine's actual calculus from an alternative encoding in the unary-theory archive. Sections 5–7 give the additional research, its computational scope, and its limits. Section 10 describes the accompanying offline artifacts.

Contents

1 Question, conventions, and acceptance criterion **3**
1.1 Four evidence levels . . . 3

2 What the supplied proposals actually represent **4**
2.1 The original five U209 machines . . . 4
2.2 The five U208 packets at the two later commits . . . 4
2.3 Stable representatives and fresh checks . . . 5

3 The existing 209-to-208 saving, in full **5**
3.1 Why the conditional history matters . . . 6
3.2 Four finite paths . . . 6

4 Which theory and enumerator the machines implement **7**
4.1 Unary weak covering . . . 7
4.2 The native calculus is not the positive dyadic alternative . . . 7
4.3 The interior startup and protected stack . . . 8

5 A finite-model obstruction to identifying source and output **8**

6 Exact exhaustion of an axiom-parameter permutation family **9**
6.1 Why the enumeration is small and auditable . . . 9

7 Searches for an additional physical deletion **10**
7.1 Both outgoing rows may change . . . 10
7.2 Source-conditioned contexts and stronger register facts . . . 11
7.3 A known attractive failure: grammar sharing without runtime preservation . . . 12

8 What the research does and does not establish **12**

9 Further research questions **12**

10 Artifact contents and reproduction **13**

1 Question, conventions, and acceptance criterion

The original question asked for a machine with fewer than 209 states, after comparing several constructions based on the proposed unary theory U. The subsequently supplied commits already contain 208-state constructions. The research target of this dossier is therefore **at most 207 working states**. Reproducing a supplied U208 construction does not meet that revised target, and a state renaming does not constitute a further saving.

Throughout, a machine has one two-way infinite tape, alphabet $\{0, 1\}$, blank symbol 0, head coordinate 0 at initialization, and an all-zero initial tape. Every transition writes one symbol and moves exactly one position left or right. The halt label is separate and is not a working state. Initialization counts. Sparse native labels such as 211 or 302 are names; they do not indicate the number of states. A row

$$q \quad b \quad w \quad D \quad r$$

means: in state q scanning b, write w, move in direction D, and enter r. Entry into the halt label halts after the write and move.

An acceptable additional saving needs more than a syntactically shorter source program or a finite blank-tape run. The intended endpoint is

$$\mathsf{PA} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \iff \mathrm{Con}(\mathsf{PA}). \tag{1}$$

A local reduction can establish this by proving, within PA, equivalence with an already calibrated parent. A changed source program instead needs a source-level soundness and completeness argument, the corresponding literal machine implementation, and a valid blank initialization. The distinction matters because source decoding and arithmetic operations share physical states in these compact machines.

1.1 Four evidence levels

The archive distinguishes four levels of evidence:

1. *Exact finite data*: table totality, target closure, state count, renaming, hashes, and rooted graph isomorphism.
2. *Finite proof certificates*: bounded predecessor invariants and whole-tape local simulation identities, or inductive symbolic runtime contracts with arbitrary natural-number parameters.
3. *Inherited mathematical calibration*: enumeration of the native calculus and its consistency equivalence with U and PA.
4. *Proof-assistant certification*: a final theorem checked in a kernel against the exact table and exact theory specifications.

The supplied U208 packets and this work provide evidence at the first three levels, with different portions newly checked here. They do not export a new end-to-end PA, Lean, Rocq, or MM0 proof object for a smaller U machine.

The cited K215 developments are a useful standard of comparison. The ProveIt verification directory describes external Lean and Rocq endpoint theorems for its exact K215 table; the separate `k215-pa` repository describes an internal PA theorem in Metamath Zero. The m254 directory provides a related Coq development for a different table and theory. Those are valuable precedents, but their certificates do not automatically apply when the searched theory or literal source program is changed [6, 5, 8].

2 What the supplied proposals actually represent

2.1 The original five U209 machines

There are five code-bearing U209 packets and one additional PDF-only packet. The following classification is established by a forced state bijection, not by similarity of descriptions or by a short execution trace [3].

U209 packet	Class	Depth	Construction emphasis
Plain / unqualified	A	42	A sequence of closed decoder rotations from U212; logical obstruction to deleting a propositional row.
Composed	A	42	The same final machine with different historical labels and a continuation-transport presentation.
Compatible Folds	A	42	Staged emitter and combined reductions, with separate local and source audits.
Joint	A	42	Direct allocation of the final selector region and a compact deterministic recipe.
Compatibility	B	41	A genuinely different allocation across a shared runtime helper.

Compatible Folds and Joint have byte-identical native tables, dense tables, and final source maps. Plain and Composed are state renamings of this same machine. The extra Joint file whose name ends in `(1).zip` contains a PDF and supplies no sixth machine. Compatibility is a second rooted action-labeled graph. Thus there are **two**, not five, independent literal implementations among the original code packets.

Both source layouts have the same ordered word of 3,909 primitive operations and the same 5,083 guarded ordinal control edges. They differ in their binary instruction addresses: 720 addresses differ. Class A has maximum address length 42; Class B has maximum length 41. Both have maximum zero run 6. The source tree has 3,908 internal prefix interfaces.

2.2 The five U208 packets at the two later commits

The first later commit adds Conditional Context, Entry Context, and History. The newest requested commit adds an unqualified U208 ConPA packet and a U208 research release. Together they present nine tables, but still only two rooted machine classes [2, 1]:

U208 packet	Main class	What its presentation adds
Conditional Context	A	A Boolean invariant audit and local finite-path checks; also supplies Class B.
Entry Context	A	Complete incoming-edge inventories, four explicit blocks, and a discovery program; also supplies Class B.
History	A	Keeps the other native label in the fold, uses Plain labels, and includes a rejected grammar shortcut; also supplies Class B.
Unqualified U208 ConPA	A	Keeps native 172 in Joint labels and provides an independent comparison and fold presentation.
U208 research release	B	Uses Compatibility as primary and Class A as `variant_plain`; supplies full final-table receipts for both.

Conditional and Entry have byte-identical primary tables. History and the unqualified packet keep state 172 where Entry keeps 211; the surrounding historical labels also differ in History. The newest research primary has exactly the same native transition dictionary as Entry's Compatibility alternative, although serialization changes the file hash. Its secondary table is a renaming of Entry's primary.

This distinction between byte identity and machine identity is essential. Different line spacing can change SHA-256 without changing one transition. Conversely, two equally sized tables can differ in a way that prevents any state bijection. The comparison checker starts at state 0, traverses the read-0 edge before the read-1 edge, preserves write and movement labels, and checks the unique forced correspondence at every reached state. All working states are graph-reachable in these tables. This is reachability in the transition graph, not a claim that the blank computation visits all of them.

The practical conclusion is exact: **the five U208 packages spend the same additional state identification**. There is no second independent deletion to obtain by combining their titles or patches.

2.3 Stable representatives and fresh checks

We use Entry's native labels for Class A in this report. Its SHA-256 is

`b348a466a2c316511e0037ea3824de29d4c6ad37338404fbc98a3968fdb25135`.

The Class B representative in the same packet has native SHA-256

`5f936da0cf393548cdd1eeebcaf7490d7a42f4ccca115f865a7bcc045463eb52`.

The archive manifest binds every other representative, dense table, source map, and upstream archive identity. Full maps are machine-readable rather than left implicit in prose.

All original U209 quick replays passed in disposable copies, and a complete universal runtime replay was performed for one representative of each U209 class. Each newer U208 packet's finite/quick reproduction also passed. The two final U208 classes were then freshly checked at the universal runtime layer:

Literal table	Guarded contracts	Symbolic inferences	Rows covered
U209 Class A	5,084	1,250,049	418/418
U209 Class B	5,084	1,248,748	418/418
U208 Class A	5,084	1,251,865	416/416
U208 Class B	5,084	1,250,564	416/416

These are inductive symbolic contracts over arbitrary register values, not merely 5,084 sampled finite executions. Their validity still depends on the stated symbolic inference rules and mathematical interpretation of the frame; a Python replay is not a proof-assistant kernel. The exact 48,140-step blank startup is also preserved. Its canonical frame has 120 one-cells and 43 spare registers, each of value 1.

3 The existing 209-to-208 saving, in full

Let P be the Joint U209 table and M be Entry's U208 table. Define the state projection $\rho(172) = 211$, fixing all other labels including start and halt [2]. Delete the two outgoing rows of state 172. Only two retained rows change:

Row	Parent action	Child action
116/0	0 R 172	0 R 211
211/0	0 R 157	0 L 217

The 211/1 action remains 1 R 184. The source map is unchanged. Keeping 172 and deleting 211 gives a renamed version of the same child, with a different number of text rows listed as edits.

3.1 Why the conditional history matters

Put the head at position 0. Two facts hold at every relevant old-machine configuration reached from state 0, even on an arbitrary initial tape:

$$q = 172 \implies (t(-2), t(-1)) = (1, 0), \tag{2}$$

$$q = 211 \ \wedge \ t(0) = 0 \implies t(-1) = 1. \tag{3}$$

For (2), the only entry into 172 is 116/0, and the only entry into 116 is 215/1. These two right moves write the predecessor word 10. Neither state is initial.

For (3), the complete entry list is $163/0 : 0L211$ and $163/1 : 1R211$. The second directly writes the required preceding 1. In the first case, all entries to 163 are rightward:

$$132/1 : 1R163, \qquad 199/1 : 1R163, \qquad 217/0 : 0R163.$$

The first two would make the zero currently scanned by 211 equal to the one just written, a contradiction. Thus entry was via 217/0. The only entry into 217 is $216/1 : 1R217$, which supplies the required left-hand 1. The guard is conditional on the current bit; replacing it by an unconditional single-cell fact loses the information that makes the fold possible.

3.2 Four finite paths

Write q_h for a control state and head coordinate. In the old 172/0 case the known cells are $(-2, -1, 0) = (1, 0, 0)$, and the child takes

$$\begin{aligned} 211_0 \to 217_{-1} \to 163_0 \to 211_{-1} \to 217_{-2} \\ \to 215_{-1} \to 157_0 \to 67_{-1}. \end{aligned}$$

In the old 211/0 case, $(-1, 0) = (1, 0)$ and the child takes

$$211_0 \to 217_{-1} \to 215_0 \to 157_1.$$

Both read-1 cases take the shared action to 184_1. Every step in these paths writes back the bit read, and no unknown exterior cell is inspected. The entire tape, not just a window, is preserved. The endpoints are exactly the projected old endpoints. Intermediate returns to 211 use its new transition; they are not assumed to satisfy the old-state meaning.

All 414 other old rows commute with ρ. By induction on old steps, each old run is simulated by positive blocks of length 1, 3, or 7. For finite halting runs,

$$T_M = T_P + 6N_P(172, 0) + 2N_P(211, 0), \qquad T_P \le T_M \le 7T_P. \tag{4}$$

Halting reflection can be expressed without infinite objects: if the child halts by time S while the parent has not halted within S steps, the block induction constructs a nonhalting child prefix of length at least S, a contradiction. This is a direct PA-formalizable finite argument.

The Class B proof is the corresponding relabeling of the local helpers. It does not use an additional independent invariant or save another state.

4 Which theory and enumerator the machines implement

4.1 Unary weak covering

The proposed theory U uses an equality-free membership language. For each formula $\varphi(x)$ with no other free variable, it contains

$$\forall a\, \exists b\, \big[\forall x(x \in b \to \varphi(x)) \ \wedge\ \forall x\big(\mathrm{Sub}(x,a) \to (\varphi(x) \to x \in b)\big)\big]\,, \tag{5}$$

where $\mathrm{Sub}(x,a)$ abbreviates $\forall z(z \in x \to z \in a)$. This is weak covering: the selected subsets of a must be included, and all members of b satisfy the matrix. It is not an unrestricted exact separation axiom with arbitrary parameters. The distinction is central to the claimed minimality of the theory [4].

The unary archive gives an ordinary mathematical argument for proof-code translations in both directions between the appropriate consistency statements. The lower interpretation needs more than a sentence saying "finite ordinals exist." In particular:

1. Equality is represented by equality of member profiles. Substitution in the member position requires saturated containing objects; it is not automatic from the definition of coextensionality.
2. Numerical representatives have internal order and predecessor properties. Induction is established before successor totality, so that successors are not silently assumed while proving them.
3. Parameter induction is obtained by using the genuinely unary matrix $Q(v) = \forall \vec{p}(A_\theta(\vec{p}) \to \theta(v, \vec{p}))$, where A_θ expresses the base and step premises.
4. Least failed numerical tuples are bound inside unary formulas to obtain bounded numerical comprehension with its support condition. This does not introduce a uniform truth predicate.
5. Successors, powers of numerical sources, pair codes, and canonical graphs for addition and multiplication are built only after the required induction and comprehension statements.

The inspected proof supplies these dependencies. This audit found no concrete parameter violation or circularity in those steps. It does not replace them by a newly implemented general proof translator.

For the upper interpretation, use $x \in a$ as the bit predicate $\mathrm{Bit}(a,x)$. Translated subsethood bounds the possible x; formula-specific PA induction forms the required finite collection. This is compatible with unbounded formula complexity because a separate PA proof is produced for each fixed formula.

4.2 The native calculus is not the positive dyadic alternative

The actual U209/U208 source retains a four-tag numerical calculus. With the constructor used by the code,

$$P(x,y) = \frac{(x+y)(x+y+1)}{2} + y,$$

its tags are implication $4P$, membership $4P+1$, quantification $4P+2$, and auxiliary equality $4P+3$. Code 0 is a normalized closed truth alias; target code 1 is diagonal membership, interpreted as false under global irreflexivity. The pairing convention must be taken from the executable constructor, rather than guessed from the phrase "dual Cantor pairing."

The separate positive dyadic stream in the unary archive uses different tags and target code 0. It is an alternative design, not the backend of these literal tables. A short expression in that encoding is consequently not a measured physical state saving for U208.

The machine-facing cover emitter allows matrix parameters. Its calibration runs through unrestricted weak covering W with global irreflexivity IR, and then through the unary theory and arithmetic. The machine does not execute a free-variable test enforcing the unary restriction in (5). In the inherited role-reassigned emitter, matrix code/output label is a, source label is b, and the tested label is c. The gates $c \neq a$ and $c \neq b$ have a logical role; they cannot be discarded on the assumption that every collision is a harmless alpha-renaming.

The admitted case $a = b$ is sound, but it supplies an empty-source cover, not a general cover. The next section turns this observation into an obstruction to one tempting global simplification.

4.3 The interior startup and protected stack

The compact initializer enters an interior axiom-emitter continuation. Its stored arithmetic value 1 is not an assumed theorem of contradiction. The retained source bridge takes 141 source steps from the specified interior store to the canonical frame, and the suffix argument constructs nine genuine theorem heads above an opaque numerical tail 4.

That tail would be unsafe if exposed: $4 = P(1, 1)$ and $1 = P(1, 0)$. Soundness therefore depends on the scheduler's protected-tail argument. The inherited conservative stack weights are $S = -1$, $MP = 0$, and $AX = 10$; the maximum initial prefix debt is two, leaving margin seven above the unsafe tail. A ghost list length handles aliases such as $P(0, 0) = 0$. The completeness compiler works above arbitrary leftover stacks after the counter/cache reset, and successful candidate codes are cofinal.

Any rearrangement of axiom builders must preserve the necessary head count, scratch-register contracts, and the interior suffix proof, or replace them explicitly. Ordinary register-machine extensional equivalence without these boundary conditions is insufficient.

5 A finite-model obstruction to identifying source and output

This section gives an additional mathematical result, distinct from the already supplied gate-deletion and propositional-row counterexamples. It rules out fixing the independent source role equal to the output role throughout the unchanged cover mechanism.

Define $\mathrm{Emp}(x)$ by $\forall z \neg(z \in x)$. Let C consist of global irreflexivity and all instances, with arbitrary additional matrix parameters, of

$$\exists b \forall x \left[(x \in b \to \varphi(x, \vec{p})) \ \wedge \ (\mathrm{Emp}(x) \to (\varphi(x, \vec{p}) \to x \in b)) \right], \tag{6}$$

with the usual freshness of the bound output variable. These are the empty-source covers obtained from the captured subset guard in the collision case. Under IR, the captured guard $\forall b(b \in x \to b \in b)$ is equivalent to $\mathrm{Emp}(x)$.

Theorem 5.1. *The fixed two-element structure with universe $\{e, u\}$ and sole membership edge $e \in u$ satisfies every instance of (6) and global irreflexivity. PA proves* $\mathrm{Con}(\mathsf{C})$.

Proof. Both objects are irreflexive and e is the only empty object. Fix any formula and any assignment to its parameters. If $\varphi(e, \vec{p})$ is false, choose output e. The upper clause is vacuous, and no empty object both satisfies the matrix and requires membership. If $\varphi(e, \vec{p})$ is true, choose output u. Its sole member is e, so the upper clause holds, and it contains the unique empty object required by the lower clause. The value of $\varphi(u, \vec{p})$ does not affect either argument.

Satisfaction of coded first-order formulas in this fixed finite structure is primitive recursive. PA verifies the preceding witness choice for every coded axiom instance and verifies soundness by induction on finite proofs. It follows that PA proves that there is no C-proof of a contradiction. □

Corollary 5.2. *If PA is consistent, this globally collapsed cover theory cannot have a PA-provable consistency biconditional with PA.*

Proof. Such a biconditional, together with Theorem 5.1, would give $\mathsf{PA} \vdash \mathrm{Con}(\mathsf{PA})$, contrary to the second incompleteness theorem for the usual arithmetized proof predicate. □

The structure fails genuine full covering already for the true matrix at source u. Both e and u are subsets of u, so a full cover would have to contain both; neither object does. Thus the loss of strength is visible inside the two-element structure.

The executable certificate checks all four possible extensions of the matrix on $\{e, u\}$ and the displayed failure of full covering. It is a finite corroboration of the model. The theorem for arbitrarily complex formulas comes from the semantic argument and proof induction, not from mistaking four matrix extensions for an enumeration of formulas. This obstruction leaves open different encodings, additional schemas, or a different source representation.

6 Exact exhaustion of an axiom-parameter permutation family

There is a sound way to vary the arithmetic emitter without changing its logical axiom set. In each of the seven complete unrestricted schema rows $(G, \rho_6, \rho_5, \rho_4, \rho_3, \rho_2, \rho_1)$, independently permute the three enumerated parameters a, b, c. A permutation is a bijection of $\mathbb{N}^3$, so it preserves the set of instances emitted by that row. It can nevertheless change the sharing available in a source decoder DAG.

The seven ordinal source regions, with half-open endpoints, are

$$[478, 887), [887, 1170), [1170, 1466), [1466, 1765),$$

$$[1765, 2061), [2061, 2411), [2411, 2642).$$

For this experiment all source association and every primitive outside the selector substitutions are fixed. The cost is the number of distinct structural subtrees, including primitive leaves, in the complete expanded source. It is *not* the literal TM state count.

Proposition 6.1 (Finite-family result)**.** *Among all* $6^7 = 279{,}936$ *independent parameter permutations in the specified fixed-association family, the minimum source DAG size is 197. Exactly four choices attain that minimum.*

The four minimizers are the original assignment; swapping b, c in ρ_6; simultaneously swapping b, c in ρ_4 and ρ_2; and performing both changes. This is an exact finite exhaustion, not a random search or a coordinate-descent conclusion.

6.1 Why the enumeration is small and auditable

The checker interns primitive trees structurally. It separates 146 constant nodes, 39 dynamic nodes whose sizes prevent collisions with all other nodes, and 49 possibly colliding dynamic templates. In this instance each contested template depends on at most one row's permutation. Its possible values can be precomputed. Each full choice is then evaluated by a union of finite bitsets, with 208 IDs in the lookup universe.

The key exactness condition is that every selector macro's interior already occurs outside the editable rows. Adding their variants to the constant part therefore does not overcount a node that might be absent for a particular assignment. The checker verifies this condition and the size/dependency partition. The retained run also cross-checked 343 full tree expansions against the decomposition. The portable artifact exposes the full histogram and all four minimizers.

DAG	Count	DAG	Count	DAG	Count	DAG	Count
197	4	198	8	199	32	200	157
201	455	202	1,159	203	2,787	204	5,630
205	9,696	206	14,899	207	22,160	208	34,080
209	46,828	210	53,711	211	49,478	212	30,484
213	8,368						

This closes one specific source-DAG optimization family. It does not prove that its variants require 208 physical states: contextual allocation can reuse runtime states in ways that are not measured by a pure subtree DAG. Conversely, a smaller DAG would not by itself prove a smaller correct machine. Some tied variants failed protected allocation or a subsequent universal runtime contract, illustrating the distinction.

7 Searches for an additional physical deletion

7.1 Both outgoing rows may change

The supplied discovery programs mainly exploit pairs with a compatible row and modify the other row. We broadened the search to unordered pairs of U208 states with *both* keeper transitions freely synthesized. After the main run, the initially excluded pairs involving start state 0 were also tested, always keeping label 0 as the projected start.

For each class the resulting census is

$$\binom{208}{2} = 21{,}528 \text{ pairs}, \qquad 35{,}650{,}368 \text{ partial single-row action trials.}$$

A candidate action chooses its written bit, direction, and any retained working-state target, including the keeper. The removed state is projected to the keeper in all other targets. The new halt label is not used as a synthesized target in this search.

Old reachable contexts are overapproximated by disjunctive cylinders obtained from depth-12 transition histories, with initial state 0 injected at each stage. Keeping the cylinders separate preserves correlations between the current bit and neighboring cells; taking only the intersection of their fixed bits would lose precisely the information used by the existing U208 fold. Each cylinder is given three deterministic pseudorandom exterior completions. The candidate is required to reproduce one old step within at most 71 new steps, with the same head and whole concrete tape at the synchronization boundary and the projected control state.

The implementation prunes a partially specified row only after a failure that occurs before the other keeper row is needed. It then screens every surviving pair of rows against all retained cases. The same procedure on the U209 parent recovers the supplied saving exactly:

$$\text{delete } 211, \quad \text{keep } 172, \quad 172/0 = 0L217, \quad 172/1 = 1R184.$$

This positive control checks that the search can rediscover the important conditional-history fold and is not vacuously rejecting every candidate.

No pair survived on either U208 class, including the 207 pairs involving state 0. This is a reproducible negative result for the stated rejection screen. The history abstraction includes configurations not established to be reachable by the actual proof program; finite exterior completions are only rejection tests; and the 71-step bound and one-old-step boundary are substantial restrictions. The result is not a lower bound on machines for Con(PA), or even a proof that every semantically valid quotient of U208 has been excluded.

7.2 Source-conditioned contexts and stronger register facts

The universal runtime was instrumented to harvest local tape windows at all primitive contracts. A radius-4 harvest produced 21,358 local windows; compressed scan/list entries were initially widened conservatively. The source-conditioned screening tested 42,642 directed aliases and 370,944 one-row mutations across 448 compatible oriented pairs, first with step bound 31, then with depth-10 history intersection and bound 45. No survivor was obtained. These contexts add source-specific restrictions; they are not uniformly stronger than arbitrary initial-tape histories, because some compressed loops were widened. Intersecting the two families does strengthen the approximation. The same acceptance boundary still applies.

Further abstract interpretation used all 512 zero/positive patterns of the nine primary registers, retaining correlations between registers. The resulting source invariant contains 278,384 reachable abstract configurations and strengthens 1,649 guarded branch contracts. A separate interval analysis adds exact-small-value and lower-bound information. These are inductive facts about source execution, not observations from a finite native trace.

The spare registers supply another useful invariant. The exact startup has 43 spares of value 1, an increment appends a spare of value 1, and the remaining primitive contracts preserve the spare list. Thus every spare can be fixed to 1 throughout a source-reachable run. A generic runtime contract allowing arbitrary spare values is stronger than necessary. Restricting to this closed invariant can sharpen discovery without restricting the unbounded primary-register search.

With this invariant, precise unary-scan interiors, and the interval facts, the restricted runtime rederives 5,079 admissible contracts with 1,243,924 symbolic inferences and still covers all 416 rows. Its radius-4 context set has 12,490 windows. Intersecting it with depth-12 history gives 51,279 disjunctive cylinders. The full two-row screen was repeated on all 21,528 Class A pairs using these stronger contexts and the same 71-step bound; it again produced no survivor. Its 169 checkpointed parts sum to 35,650,368 partial action trials. This second run changes the reachable context approximation, not the boundary relation or the finite-screen acceptance status.

One further family deletes a state with uniformly directed incoming moves and flips those moves when redirecting them. The configuration relation places the projected head two cells away at the deleted-state boundary. Allowing an arbitrary single keeper-row mutation screened 36,639 ordered pairs and 60,674,184 actions on each U208 class, with depth-12 contexts and a 71-step bound; neither class produced a survivor.

A completed Class A block experiment permits both old and new machines to take positive blocks, each of length at most 31. Deletions were restricted to $\{163, 211, 217, 215, 157, 184, 67, 148, 186\}$; every distinct noninitial keeper and one arbitrary keeper-row replacement were considered. Depth-9 guards gave 1,854 ordered pairs and 3,070,224 assignments. Twenty finite-screen survivors reached the symbolic stage; none was accepted within its 4,096 branch-processing budget per guard. This does not assert that every such transformation is semantically invalid. Incomplete source-conditioned extensions are marked as partial and excluded from completed censuses.

At the source level, all 5,040 orders of the seven logical builders were also tested with a particular bounded association heuristic: five closed slots, factor counts $(1, 6, 6, 1, 16)$, and beam width 1 within each slot. The best produced DAG had 199 nodes, compared with the original 197. This exhausts the orders under that heuristic, not the association space. A tied 197-node parameter variant obtained a prefix-valid 208-state allocation after an extra protection filter was relaxed, but its fresh universal runtime check failed at source site 23, zero branch of decrement register 3, after 31 successful cases. It is not an accepted new machine.

7.3 A known attractive failure: grammar sharing without runtime preservation

The supplied newer packets already diagnose the following apparent saving. It was also encountered independently during this work and is included as a regression example, not as a novelty claim. In Joint native notation,

$$155 = (85, 150), \quad 85 = (98, 12), \qquad 52 = (154, 150), \quad 154 = (166, 12).$$

Reassociate each $((A, B), C)$ into $(A, (B, C))$. This makes the two inner states have children $(12, 150)$, apparently allowing one to be deleted. The primitive word and ordinal source control graph can be preserved. The physical runtime nonetheless changes because it uses the same graph through a backward entry into the decoder.

An exact seventeen-step prefix witness begins at dispatcher state 84, head 0, left marker 1, and known instruction prefix `0011001000001`. Exterior cells remain opaque. The original table first crosses the end of that prefix in state 42 with head 13 and no tape change. The reassociated table reaches state 25 with head 13 and changes cell 12 from 1 to 0. The first twelve transitions agree. The hidden route includes $76 \to 152 \to 159 \to 118 \to 155 \to 85$.

The portable witness checker does not import the symbolic engine and fails on any unknown exterior read. It confirms a concrete failure of the claimed prefix contract. It does not determine the rejected table's eventual blank behavior. In particular, it must not be advertised as a 207-state candidate merely because combining it with the existing fold produces a table with that number of labels.

Closed-rotation searches and context-aware allocation attempts were also performed. Their useful role is to generate candidates; a source control graph check alone is never the final acceptance condition. The audit trail retains negative examples so that this distinction is testable.

8 What the research does and does not establish

The exact comparison identifies two rooted graph classes at each state count: four literal graphs in total, belonging to two source-layout lineages. It tracks their labels and verifies the same physical saving in the later packets. The final U208 runtime contracts have been freshly rederived for both classes. The new finite-model theorem rules out one natural logical simplification, and the exhaustive permutation census gives a precise minimum for one substantial source family.

None of those statements proves that 208 is minimal. State counts are the end of a chain of choices: axioms, formula coding, enumeration program, source association, contextual allocation, tape representation, and local machine reductions. A negative result at one level does not bound the other levels. Nor does a broad heuristic search establish that an unexamined configuration relation cannot save a state.

The search code therefore records assumptions, bounds, and positive controls alongside outcomes. A generated table that lacks a proof of its intended behavior is kept in rejected research data rather than silently promoted to a consistency machine. The requested construction with at most 207 working states remains unresolved in this investigation.

9 Further research questions

1. **Can source layout and helper behavior be synthesized together?** The repeated grammar failure shows that a state should carry a family of entry contracts, including backward arithmetic entries. An allocator could optimize these contracts jointly with source association instead of first optimizing a pure decoder and later discovering the conflict.

2. **Can the configuration relation change economically?** Two machines need not synchronize after every old transition with identical head coordinates and tape. Small fixed head offsets, temporary marker changes, or synchronization at source boundaries may permit reductions excluded by the present local search. The associated proof should establish positive progress and halting reflection explicitly.
3. **Can the all-spares-one invariant replace a generic list engine?** A proof specialized to fixed spare blocks might permit a shorter scanner or delimiter mechanism while leaving the primary registers unbounded. The whole initialization and append invariant must remain part of the theorem.
4. **Can axiom-builder order and association outperform parameter permutations?** The 279,936-case result fixes association. Reordering complete builders preserves the emitted axiom set, but interacts with scratch-state reuse and the protected-tail argument. A physical saving needs a measured allocation and final runtime proof.
5. **Can the positive dyadic calculus be compiled completely?** Its alternative syntax may simplify formula construction, but the full cost includes proof enumeration, vacuity/freshness handling, substitution if used, and blank initialization. Comparing only formula sizes is inconclusive.
6. **Can the U/PA calibration be turned into an explicit proof translator?** A proof-code implementation would separate the mathematical strength question from changes to the machine backend and would provide a clear target for the MM0 or other small-kernel route.

10 Artifact contents and reproduction

The dossier contains editable LaTeX, its compiled PDF, exact baseline tables and source snapshots, fresh receipts, the collision-model checker, the exhaustive permutation census, and bounded-search code. Original Entry Context and U208 research release archives retain the runtime engines and mathematical lineage needed for offline replay. Their original notices and licenses remain attached.

Run from the extracted release root with Python 3.10 or later:

```
python3 -B verify.py --output /tmp/conpa-finite.json
python3 -B verify.py --full-runtime --output /tmp/conpa-full.json
```

The default verifier checks payload integrity, compares the literal machine snapshots, reproduces the finite collision model and regression witness, exhausts all 279,936 parameter choices, and replays the supplied finite baseline certificates. It explicitly skips the full runtime. The second command rederives all 5,084 universal contracts for each of the two U208 classes and can take several minutes. Longer discovery searches are separate; for example, with a C++17 compiler:

```
python3 research/two_row/run.py --class control --output /tmp/conpa-control
python3 research/two_row/run.py --class A --output /tmp/conpa-search-A
python3 research/two_row/run.py --class B --output /tmp/conpa-search-B
```

Each command states whether it is a mathematical certificate replay or a bounded rejection screen. Python's optimized mode is supported by the small independent checkers; inherited full symbolic engines require assertions.

`MANIFEST.json` binds every other payload file; verification uses temporary directories. Run `pdflatex` twice on `report.tex` from `paper/` to rebuild the PDF. Rebuilding can change PDF metadata while the fixed machine tables remain identical.

References

[1] VladimirReshetnikov/ProveIt, *Incoming research at the latest requested commit*, https://github.com/VladimirReshetnikov/ProveIt/tree/34aaab66fc5dc834863ed00ea84823c93b8c5b1e/docs/incoming. Includes the five U208 packets and their retained attribution, proofs, transition tables, and executable evidence.

[2] VladimirReshetnikov/ProveIt, *Conditional Context, Entry Context, and History U208 packets*, https://github.com/VladimirReshetnikov/ProveIt/tree/ef5a23e04b0285e321ceec5f4b329223cee0ab57/docs/incoming.

[3] VladimirReshetnikov/ProveIt, *Original U209 incoming proposals*, https://github.com/VladimirReshetnikov/ProveIt/tree/fe8b3a2d1db0033113e986590f79ecba3e5ef500/docs/incoming.

[4] *PA unary power cover*, supplied archive, https://github.com/VladimirReshetnikov/ProveIt/blob/34aaab66fc5dc834863ed00ea84823c93b8c5b1e/docs/incoming/pa_unary_power_cover.zip. The report uses the archive's unary-core, unary-arithmetic, upper-transport, and positive-interface arguments.

[5] Elliot Glazer, *K215 halts iff ZFC is inconsistent (Metamath Zero, PA)*, https://github.com/elliotglazer/k215-pa. Repository specification and verification receipt, consulted 29 October 2026.

[6] VladimirReshetnikov/ProveIt, *Formal correctness of the exact 215-state machine*, https://github.com/VladimirReshetnikov/ProveIt/tree/34aaab66fc5dc834863ed00ea84823c93b8c5b1e/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215.

[7] Elliot Glazer, *Theory K and the K230 development*, https://github.com/elliotglazer/k230-rocq#the-theory-k.

[8] jeff87654/busy-beaver, *A verified 254-state Turing machine that halts iff ZFC is inconsistent*, https://github.com/jeff87654/busy-beaver/tree/zf-254/ZF/m254.